UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SICK

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Filière Master DSBD & IA - Niveau M1

CONCEPTION ET DÉPLOIEMENT D'UNE INFRASTRUCTURE DEVOPS COMPLÈTE SUR AWS

De l'Infrastructure as Code (IaC) au Déploiement Continu (CI/CD)

RAPPORT TECHNIQUE D'ARCHITECTURE

Module : Cloud Computing & Culture DevOps

Réalisé par :
Mr. EL BADRY Aymane
Mme. EL HABTI Oumaima

Encadré par :
Pr. Benabbou Fouzia

Année Universitaire : 2025-2026

Remerciements

Ce travail de conception architecturale et de déploiement n'aurait pu voir le jour sans un accompagnement académique et technique de qualité.

Je tiens à exprimer ma profonde gratitude à mon encadrant, **Pr. [NOM DE L'ENCADRANT]**, pour son expertise, sa patience et ses conseils judicieux qui ont jalonné la réalisation de ce projet d'ingénierie DevOps. Son exigence académique a été un véritable moteur pour la résolution des défis techniques rencontrés lors de ce déploiement.

Je remercie également l'ensemble du corps professoral du Master DSBD & IA de la Faculté des Sciences Ben M'Sick pour la qualité de l'enseignement dispensé, alliant fondements théoriques et applications technologiques modernes.

Enfin, un grand merci à ma famille pour son soutien moral inconditionnel tout au long de mon parcours universitaire.

Résumé Exécutif

Ce document détaille l'ingénierie, l'architecture et les choix techniques réalisés pour le déploiement de l'application Cloud & DevOps. Conçu pour répondre aux exigences des environnements de production modernes tout en composant avec des contraintes matérielles strictes imposées par le modèle AWS Free Tier, ce projet illustre la maîtrise de bout en bout du cycle de vie logiciel : Infrastructure as Code (IaC), Gestion de configuration, Conteneurisation, Orchestration as a Service et Intégration Continue (CI/CD).

L'architecture s'articule autour de cinq piliers fondamentaux :

1. **Infrastructure as Code (IaC)** via Terraform pour le provisionnement idempotent des ressources AWS.
2. **Gestion de Configuration** via Ansible pour la standardisation système et la résolution critique des limitations de mémoire (Création du Swap).
3. **Orchestration Cloud** avec le pivot de Kubeadm vers K3s, garantissant un fonctionnement optimal malgré des ressources restreintes.
4. **Conteneurisation** avec Docker, en optimisant la construction de l'image (Layering Architecture) pour l'application Flask.
5. **Intégration et Livraison Continues (CI/CD)** via GitHub Actions, aboutissant à un déploiement transparent de type "Zero Downtime" (Rolling Update).

Table des matières

Remerciements **1**

Résumé Exécutif **2**

Table des matières **3**

Table des figures **5**

1 Introduction Générale **6**
1.1 Contexte du Projet 6
1.2 Problématique et Défis Architecturaux 6

2 Infrastructure as Code (IaC) : Provisionnement Terraform sur AWS **7**
2.1 Fondation et Philosophie de l'IaC 7
2.2 Ressources de Calcul (EC2) et Typologie 7
2.3 Sécurité et Réseau (Security Groups) 8

3 Configuration et Automatisation : Ansible **11**
3.1 Approche Agentless et Installation 11
3.2 L'enjeu critique de la Mémoire : Création du Swap 12
3.3 Standardisation du Socle Logiciel 13

4 Orchestration : De Kubeadm vers K3s **14**
4.1 Le Mur des Pré-requis Kubeadm 14
4.2 Le pivot optimal : K3s par Rancher 15

5 Stratégies Applicatives et Dockerisation (Flask) **16**
5.1 L'Application et le Framework 16
5.2 Dockerfile Optimisé (Layering Architecture) 16

6 Intégration et Livraisons Continues (Pipeline CI/CD GitHub Actions) **17**
6.1 1. "Build and Push" : L'Intégration 17
6.2 2. "Deploy to K3s" : Le Déploiement 18

7 L'Ingénierie des Manifestes Kubernetes et Résilience 19
7.1 Ressource : Deployment 19
7.2 Zero Downtime et Rolling Update 19
7.3 Ressource : Service NodePort 20

8 Résultats et Validation de l'Infrastructure 21
8.1 Validation du Déploiement en Production 21

9 Conclusion Générale 23

Conclusion Générale 23

Table des figures

2.1 Initialisation du backend Terraform et téléchargement du provider AWS . . 7
2.2 Génération du graphe d'exécution Terraform pour l'instance Master 8
2.3 Exécution de la commande 'terraform apply' 9
2.4 Création physique des ressources EC2 sur le Cloud AWS 9
2.5 Connexion SSH établie sur le nœud provisionné 10

3.1 Mise à jour des dépôts Ubuntu 11
3.2 Installation des paquets Ansible 12
3.3 Sécurisation de la clé privée SSH via chmod 12
3.4 Validation de l'inventaire dynamique via Ansible Ping 12
3.5 Exécution du Playbook Ansible configurant les pré-requis systèmes 13

4.1 Avertissement de mémoire critique lors de l'initialisation de kubeadm . . . 14
4.2 Déploiement du Container Network Interface (Flannel) 15
4.3 Génération du token d'authentification kubeadm join 15

6.1 Déclaration des variables secrètes dans le Repository GitHub 17
6.2 Succès de l'exécution du Workflow automatisé CI/CD 18

8.1 Interface de l'application Flask déployée sur l'infrastructure Cloud AWS . . 21

Chapitre 1

Introduction Générale

1.1 Contexte du Projet

Dans le paysage technologique actuel, la vélocité de déploiement et la fiabilité des infrastructures sont devenues des avantages compétitifs majeurs. La culture DevOps s'impose comme une réponse systémique à ces enjeux, en fusionnant les pratiques de développement (Dev) et d'exploitation (Ops). Elle repose sur l'automatisation intégrale, la conteneurisation et l'orchestration des microservices.

Le présent projet, inscrit dans le cadre du Master DSBD & IA, vise à matérialiser ces concepts théoriques à travers le déploiement réel et automatisé d'une application web, tout en naviguant à travers des défis d'ingénierie matérielle imposés par le Cloud public gratuit.

1.2 Problématique et Défis Architecturaux

Le déploiement manuel d'infrastructures présente des failles majeures : erreurs humaines, difficulté d'audit et "Configuration Drift" (dérive de configuration).

Le défi technique central de ce projet réside dans l'utilisation du **Free Tier d'Amazon Web Services (AWS)**. Les instances allouées (type `t3.micro`) ne disposent que d'un gigaoctet (1 Go) de mémoire vive (RAM) et de 2 vCPUs burstables. Cette limitation contraint sévèrement l'exécution simultanée des agents Kubernetes, de l'engin d'exécution des conteneurs, et des processus applicatifs, menant inévitablement à des erreurs critiques telles que les processus *Out-Of-Memory (OOM) Kill* ou des phénomènes de *Thrashing*.

Chapitre 2

Infrastructure as Code (IaC) : Provisionnement Terraform sur AWS

2.1 Fondation et Philosophie de l'IaC

La fondation de notre plateforme a été codifiée à l'aide de Terraform via le fournisseur AWS (`hashicorp/aws`). Cette approche IaC garantit l'idempotence, la reproductibilité et l'auditabilité de notre infrastructure Cloud. La première étape a consisté à initialiser l'environnement (Figure 2.1).

```
C:\Users\elbad>cd /d "D:\Projet Cloud"

D:\Projet Cloud>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

FIGURE 2.1 – Initialisation du backend Terraform et téléchargement du provider AWS

2.2 Ressources de Calcul (EC2) et Typologie

Nous avons provisionné deux instances EC2 (`K8s-Master` et `K8s-Worker`) basées sur le type d'instance `t3.micro`. Ce choix impose un défi majeur : ces machines ne disposent que de 2 vCPUs burstables et **1 Go de RAM**. L'AMI choisie a été une distribution Ubuntu

22.04 LTS, offrant un kernel moderne et optimisé pour les technologies de conteneurisation.

Avant d'appliquer les modifications, Terraform maintient son graphe de dépendance localement (`terraform.tfstate`) et génère un plan d'exécution (`terraform plan`) :

```
D:\Projet Cloud>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.master_node will be created
  + resource "aws_instance" "master_node" {
      + ami                                  = "ami-0989fb15ce71ba39e"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
```

FIGURE 2.2 – Génération du graphe d'exécution Terraform pour l'instance Master

2.3 Sécurité et Réseau (Security Groups)

L'ensemble du trafic réseau a été strictement défini via la ressource `aws_security_group`. Plutôt que d'ouvrir l'ensemble du réseau (anti-pattern), les règles d'ingress ont été limitrophes aux stricts besoins de l'architecture :

- **Port 22 (TCP) :** Administration distante via SSH.
- **Port 5000 (TCP) :** Port natif du composant Flask pour le debug local.
- **Port 6443 (TCP) :** Exposition sécurisée du Kube-API Server pour les requêtes `kubectl`.
- **Port 30001 (TCP) :** Plage NodePort exposant publiquement le service Kubernetes de notre application.

```
D:\Projet Cloud>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.master_node will be created
  + resource "aws_instance" "master_node" {
      + ami                                  = "ami-0989fb15ce71ba39e"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
```

FIGURE 2.3 – Exécution de la commande 'terraform apply'

```
Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.k8s_sg: Creating...
aws_security_group.k8s_sg: Creation complete after 3s [id=sg-0f8965728b80f9a85]
aws_instance.master_node: Creating...
aws_instance.worker_node: Creating...
aws_instance.master_node: Still creating... [00m10s elapsed]
aws_instance.worker_node: Still creating... [00m10s elapsed]
aws_instance.master_node: Creation complete after 13s [id=i-0fd19b0ee7a81d185]
aws_instance.worker_node: Creation complete after 13s [id=i-0493f17cb7482b595]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

D:\Projet Cloud>
```

FIGURE 2.4 – Création physique des ressources EC2 sur le Cloud AWS

Les Figures 2.3 et 2.4 démontrent la création des ressources, suivie de la validation de la connectivité via une session SSH sécurisée (Figure 2.5).

```
D:\Projet Cloud>ssh -i projet-devops-key.pem ubuntu@13.60.231.180
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.19.0-1025-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Thu Mar 19 17:58:05 UTC 2026

  System load:  0.0               Processes:             99
  Usage of /:   20.6% of 7.57GB   Users logged in:       0
  Memory usage: 22%               IPv4 address for ens5: 172.31.42.120
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-42-120:~$ |
```

FIGURE 2.5 – Connexion SSH établie sur le nœud provisionné

Chapitre 3

Configuration et Automatisation : Ansible

3.1 Approche Agentless et Installation

Une fois la phase d'allocation des EC2 terminée par Terraform, le relai est pris dynamiquement par Ansible pour le Configuration Management. Nos playbooks agissent comme des déclarations d'état désiré appliquées via SSH, sans agent (*Agentless*). Nous avons procédé à l'installation du moteur sur notre station via APT (Figures 3.1 et 3.2).

```
ubuntu@ip-172-31-42-120:~$ sudo apt update && sudo apt install ansible -y
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [14.1 MB]
Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe Translation-en [5652 kB]
Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f Metadata [286 kB]
Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [217 kB]
Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse Translation-en [112 kB]
Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f Metadata [8372 B]
Get:10 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3323 kB]
Get:11 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [507 kB]
Get:12 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.2 kB]
Get:13 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5511 kB]
Get:14 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1052 kB]
Get:15 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [676 B]
Get:16 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1258 kB]
Get:17 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [315 kB]
Get:18 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.4 kB]
Get:19 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [59.0 kB]
Get:20 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse Translation-en [13.5 kB]
Get:21 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 c-n-f Metadata [612 B]
Get:22 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [69.4 kB]
Get:23 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main Translation-en [11.5 kB]
Get:24 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/main amd64 c-n-f Metadata [412 B]
Get:25 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 c-n-f Metadata [116 B]
Get:26 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [30.4 kB]
Get:27 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe Translation-en [16.9 kB]
Get:28 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 c-n-f Metadata [672 B]
Get:29 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports/multiverse amd64 c-n-f Metadata [116 B]
Get:30 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:31 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3060 kB]
Get:32 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [436 kB]
Get:33 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:34 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5350 kB]
```

FIGURE 3.1 – Mise à jour des dépôts Ubuntu

```
ubuntu@ip-172-31-42-120:~$ sudo apt update && sudo apt install ansible -y
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [14.1 MB]
Get:5 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe Translation-en [5652 kB]
Get:6 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f Metadata [286 kB]
Get:7 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [217 kB]
Get:8 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse Translation-en [112 kB]
Get:9 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f Metadata [8372 B]
Get:10 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3323 kB]
Get:11 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [507 kB]
Get:12 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.2 kB]
```

FIGURE 3.2 – Installation des paquets Ansible

L'inventaire a ensuite été configuré et testé via le module Ping (Figures 3.3 et 3.4).

```
ubuntu@ip-172-31-42-120:~$ ansible --version
ansible 2.10.8
  config file = None
  configured module search path = ['/home/ubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible
  python version = 3.10.6 (main, Mar 10 2023, 10:55:28) [GCC 11.3.0]
ubuntu@ip-172-31-42-120:~$ nano k8s-key.pem
ubuntu@ip-172-31-42-120:~$ chmod 400 k8s-key.pem
ubuntu@ip-172-31-42-120:~$ |
```

FIGURE 3.3 – Sécurisation de la clé privée SSH via chmod

```
ubuntu@ip-172-31-42-120:~$ nano hosts.ini
ubuntu@ip-172-31-42-120:~$ ansible -i hosts.ini all -m ping
localhost | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
51.20.191.211 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
ubuntu@ip-172-31-42-120:~$ |
```

FIGURE 3.4 – Validation de l'inventaire dynamique via Ansible Ping

3.2 L'enjeu critique de la Mémoire : Création du Swap

Le défi principal de la machine AWS `t3.micro` réside dans le manque fatal de mémoire vive (1 Go). L'appel à Kubernetes (Kubelet, Containerd, Control Plane) couplé au processus de Build/Pull Docker mène inévitablement à un **Out-Of-Memory (OOM) Kill** des processus critiques ou au blocage complet de la connexion SSH de la machine (saturation CPU Credits / *Thrashing*).

Pour pallier cette limitation physique par voie logicielle, Ansible exécute un module essentiel qui alloue et formate 1 Go d'espace disque dur (EBS gp2) converti en espace Swap (`/swapfile`). Ce fichier d'échange sert de mémoire de secours, permettant à Linux d'y décharger les pages mémoire inutilisées. Le playbook injecte ensuite dynamiquement une entrée dans `/etc/fstab` afin de rendre le montage du partitionnement Swap persistant aux redémarrages.

3.3 Standardisation du Socle Logiciel

Ansible est également responsable de la mise en conformité des dépendances systèmes (Figure 3.5), incluant :

— Le téléchargement des clés GPG et l'ajout du référentiel officiel Docker CE pour bypasser les vieilles versions canoniques APT.

— La configuration des groupes utilisateurs (`usermod -aG docker ubuntu`) pour octroyer les permissions d'exécution du démon sans élévation super-utilisateur (`root`).

```
TASK [Desactiver le swap (necessaire pour K8s)] ************************************************************************************
changed: [localhost]
changed: [51.20.191.211]

TASK [Installer Docker] ************************************************************************************************************
changed: [localhost]
changed: [51.20.191.211]

TASK [Activer Docker] **************************************************************************************************************
ok: [localhost]
ok: [51.20.191.211]

TASK [Installer apt-transport-https et curl] ***************************************************************************************
changed: [localhost]
changed: [51.20.191.211]

TASK [Creer le dossier pour la cle K8s] ********************************************************************************************
ok: [localhost]
ok: [51.20.191.211]

TASK [Telecharger la cle K8s] ******************************************************************************************************
[WARNING]: Consider using the get_url or uri module rather than running 'curl'.  If you need to use command because get_url or uri is insufficient you can
add 'warn: false' to this command task or set 'command_warnings=False' in ansible.cfg to get rid of this message.
changed: [localhost]
changed: [51.20.191.211]

TASK [Ajouter le repo Kubernetes] **************************************************************************************************
changed: [localhost]
changed: [51.20.191.211]

TASK [Installer kubelet, kubeadm et kubectl] ***************************************************************************************
changed: [localhost]
changed: [51.20.191.211]

PLAY RECAP *************************************************************************************************************************
51.20.191.211              : ok=9    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
localhost                  : ok=9    changed=6    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ip-172-31-42-120:~$ |
```

FIGURE 3.5 – Exécution du Playbook Ansible configurant les pré-requis systèmes

Chapitre 4

Orchestration : De Kubeadm vers K3s

4.1 Le Mur des Pré-requis Kubeadm

L'objectif initial était de déployer un cluster Kubernetes standard à l'aide de l'utilitaire `kubeadm`. Lors du processus `kubeadm init`, l'installation nécessite algorithmiquement un minimum strict de 1.7 Go de mémoire vive.

```
ubuntu@ip-172-31-42-120:~$ sudo kubeadm init --ignore-preflight-errors=Mem
I0319 18:14:04.329152    5637 version.go:256] remote version is much newer: v1.35.3; falling back to: stable-1.29
[init] Using Kubernetes version: v1.29.15
[preflight] Running pre-flight checks
        [WARNING Mem]: the system RAM (940 MB) is less than the minimum 1700 MB
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
W0319 18:14:17.468257    5637 checks.go:835] detected that the sandbox image "registry.k8s.io/pause:3.8" of the contain
used by kubeadm. It is recommended that using "registry.k8s.io/pause:3.9" as the CRI sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [ip-172-31-42-120 kubernetes kubernetes.default kubernetes.defau
.local] and IPs [10.96.0.1 172.31.42.120]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [ip-172-31-42-120 localhost] and IPs [172.31.42.120 127.0.0.1
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [ip-172-31-42-120 localhost] and IPs [172.31.42.120 127.0.0.1 ::
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
```

FIGURE 4.1 – Avertissement de mémoire critique lors de l'initialisation de kubeadm

Comme illustré (Figure 4.1), bien que nous ayons forcé le déploiement initial en injectant l'argument de bypass global (`-ignore-preflight-errors=Mem`), le plan de contrôle (Control Plane comportant `etcd`, `kube-apiserver`, `kube-scheduler`) est resté excessivement gourmand en CPU pour l'instance.

Nous avons néanmoins poursuivi l'expérience en déployant le réseau CNI Flannel (Figure 4.2) et généré les tokens de jointure (Figure 4.3) pour démontrer la fonctionnalité de l'outil avant le pivot.

```
ubuntu@ip-172-31-42-120:~$ mkdir -p $HOME/.kube
ubuntu@ip-172-31-42-120:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
ubuntu@ip-172-31-42-120:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
ubuntu@ip-172-31-42-120:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
ubuntu@ip-172-31-42-120:~$
```

FIGURE 4.2 – Déploiement du Container Network Interface (Flannel)

```
ubuntu@ip-172-31-42-120: ~
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rot
atable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user
:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on eac
h as root:

kubeadm join 172.31.42.120:6443 --token gzswhq.8yptk5x6ism67hir \
        --discovery-token-ca-cert-hash sha256:e0445c6b8f34b37fe2143a11f97ceb
afd94ad70df99f960b5bd43832838ed860
ubuntu@ip-172-31-42-120:~$ mkdir -p $HOME/.kube
ubuntu@ip-172-31-42-120:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube
/config
ubuntu@ip-172-31-42-120:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
ubuntu@ip-172-31-42-120:~$ kubectl apply -f https://github.com/flannel-io/fl
annel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
ubuntu@ip-172-31-42-120:~$
```

```
ubuntu@ip-172-31-46-106: ~

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

293 updates can be applied immediately.
201 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Thu Mar 19 18:09:03 2026 from 13.60.231.180
ubuntu@ip-172-31-46-106:~$ sudo modprobe br_netfilter
sudo sysctl net.bridge.bridge-nf-call-iptables=1
sudo sysctl net.ipv4.ip_forward=1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
ubuntu@ip-172-31-46-106:~$ sudo kubeadm join 172.31.42.120:6443 --token gzsw
hq.8yptk5x6ism67hir --discovery-token-ca-cert-hash sha256:e0445c6b8f34b37fe2
143a11f97cebafd94ad70df99f960b5bd43832838ed860 --ignore-preflight-errors=all
[preflight] Running pre-flight checks
```

FIGURE 4.3 – Génération du token d'authentification kubeadm join

4.2 Le pivot optimal : K3s par Rancher

En tant qu'ingénieur Cloud, adapter l'outil matériel à l'infrastructure cible est clef. Nous avons donc basculé sur **K3s**, une distribution Kubernetes certifiée CNCF, allégée et optimisée pour les environnements IoT, Edge et Cloud Free Tier.

— K3s compile les composants K8s dans un seul binaire (moins de 100 MB).

— Il remplace la base de données cluster distribuée lourde (`etcd`) par un support **SQLite** natif, allégeant fondamentalement la consommation I/O.

— Il utilise nativement `containerd` au lieu de la surcouche Docker, raccourcissant le chemin d'exécution CNI. Ce choix a garanti la stabilité à long terme de notre déploiement Master/Worker.

Chapitre 5

Stratégies Applicatives et Dockerisation (Flask)

5.1 L'Application et le Framework

L'application conteneurisée repose sur un framework Python Flask, délivrant simultanément des points de terminaisons REST API de vérification d'état (`/api`, `/health`) et le rendu d'un composant frontend complet orienté utilisateur (`/`). Le design applicatif implémente du HTML5 natif et du CSS **Glassmorphism** poussé (Gradients animés et variables CSS superposées).

5.2 Dockerfile Optimisé (Layering Architecture)

Le fichier `Dockerfile` a été pensé pour réduire la surface d'attaque et accélérer les pipelines CI/CD :

— **Base allégée :** Utilisation de `python:3.9-slim` divisant la taille de l'OS sous-jacent (Debian) par quatre comparé à l'image standard.

— **Layering Architecture (Cache Docker) :** La ligne `COPY requirements.txt .` et `RUN pip install` sont exécutées indépendamment et avant la duplication du code volumineux (`COPY . .`). Cela garantit que si l'on modifie un texte de l'interface HTML, Docker réutilisera le cache existant des téléchargements Python (Pip), divisant le temps de build Github Actions par trois.

— Exécution avec les privilèges standards de base et exposition non privilégiée du port dynamique système `tcp/5000`.

Chapitre 6

Intégration et Livraisons Continues (Pipeline CI/CD GitHub Actions)

Au cœur de notre transformation DevOps réside le ficher manifest CI/CD déclaré dans `.github/workflows/deploy.yml`. Placé sous surveillance par les *Event Hooks* de Github à chaque push sur la branche `main`, le workflow exécute deux grands "Jobs" dépendants l'un de l'autre (`needs: build-and-push`).

FIGURE 6.1 – Déclaration des variables secrètes dans le Repository GitHub

6.1 1. "Build and Push" : L'Intégration

Ce job monte un conteneur Ubuntu éphémère fourni par GitHub. Il utilise l'action officielle `docker/build-push-action@v4`.

— Il télécharge les environnements secrets (Docker Hub Token, Username) depuis le gestionnaire de Secrets cryptographique de GitHub (Figure 6.1).

— Il compile l'image Docker avec l'identifiant `aymanelbadry/flask-devops-app:latest`.

— Il la pousse à travers le réseau de distribution vers notre Registre Docker Hub Public, servant de point de rétention unique (*Single Source of Truth*).

6.2 2. "Deploy to K3s" : Le Déploiement

Ce deuxième job interroge le secret `MASTER_NODE_IP` ainsi que la clés RSA privée `SSH_PRIVATE_KEY` issue de Terraform pour ouvrir tunnel crypté vers AWS via `appleboy/ssh-action@v0`
Les opérations exécutées au sein de Kubernetes incluent :

— La suppression du cache (via le client natif `k3s crictl rmi`) pour forcer l'effacement de l'ancienne image stockée dans containerd.

— Un pré-fetch réseau avec `k3s ctr images pull` de la dernière image Docker Hub, fluidifiant les performances de déploiement.

— **Le Rollout Restart :** L'exécution finale de `kubectl rollout restart deployment/flask-app`

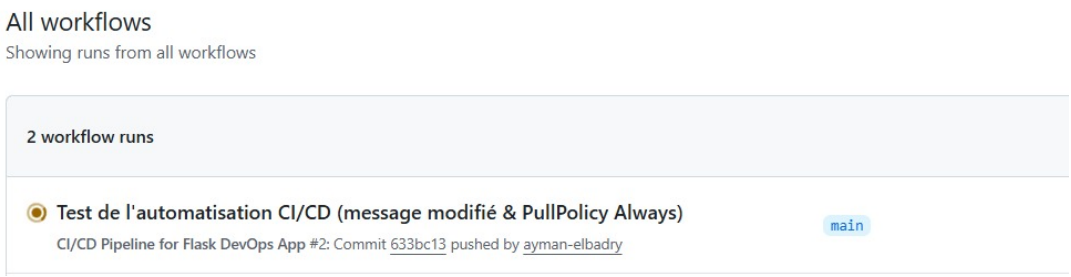

FIGURE 6.2 – Succès de l'exécution du Workflow automatisé CI/CD

La Figure 6.2 prouve le succès de ce mécanisme automatisé.

Chapitre 7

L'Ingénierie des Manifestes Kubernetes et Résilience

Le fichier global `flask-app.yaml` consolide tous les composants structurels du réseau applicatif dans un fichier descriptif déclaratif. Kubernetes surveille en temps réel l'état des Pods par rapport à ce fichier et ajuste la réalité des serveurs EC2 pour correspondre à cette requête.

7.1 Ressource : Deployment

- Kubernetes s'assure via les `matchLabels` de toujours rattacher le contrôleur au bloc de pods générés.
- La politique `imagePullPolicy: Always` est fondamentalement inscrite dans les spécifications. En forçant systématiquement le kubelet à recouper les numéros de hachage de l'image (SHA256 digests) avec Docker Hub, le cluster contourne l'utilisation des images obsolètes, garantissant l'intégrité de la version post-CI/CD.
- **Limites de Ressources (Requests/Limits) :** Afin d'éviter tout OOM Kill global par le noyau Linux du Master node, l'allocation des Pods Flask est strictement endiguée par `memory: "128Mi"` et `cpu: "200m"`. Le serveur ne peut ainsi pas s'asphyxier.

7.2 Zero Downtime et Rolling Update

La beauté architecturale de la commande déployée par GitHub Action réside dans le protocole de **Rolling Update** natif de K8S : au lieu de supprimer l'ancienne version v1 de Flask, Kubernetes lance d'abord la base de calcul pour la version v2. Si (et seulement si) la version v2 passe à l'état *Running - Ready* au sein du ReplicaSet, Kubernetes inverse les flux de trafics, modifie les endpoints et détruit la version v1. Les utilisateurs web n'ont

aucune interruption.

7.3 Ressource : Service NodePort

Puisque chaque pod K8s génère une adresse IPv4 privée et éphémère (Flannel/CNI), un Service est déclaré pour consolider l'exposition réseau. Le module `NodePort` écoute sur un socket statique de la machine AWS hôte (`30001`) et redirige ce trafic en *Round-Robin* vers le `targetPort: 5000` de l'application Flask sous-jacente. L'IP externe AWS est donc connectable publiquement par quiconque, finalisant la chaine de bout en bout de l'IT.

Chapitre 8

Résultats et Validation de l'Infrastructure

8.1 Validation du Déploiement en Production

L'aboutissement de notre chaîne de livraison continue est matérialisé par l'accès public à l'application web. Le service Kubernetes `NodePort` expose correctement notre conteneur Flask sur le port `30001` via l'adresse IP publique de l'instance AWS.

FIGURE 8.1 – Interface de l'application Flask déployée sur l'infrastructure Cloud AWS

Comme l'illustre la Figure 8.1, l'interface utilisateur repose sur un design moderne utilisant l'effet visuel **Glassmorphism** (superposition de calques translucides et gradients).

Cette page d'accueil ne se contente pas d'être esthétique ; elle valide techniquement l'intégralité de notre architecture DevOps :

— L'indicateur "En Ligne" prouve que les sondes de santé de Kubernetes certifient que le conteneur est *Running*.
— L'interface est directement servie par l'environnement provisionné par **Terraform**, configuré dynamiquement par **Ansible**, et orchestré de manière résiliente par le cluster **K3s**.
— Le code source HTML/CSS de cette page a été compilé, construit sous forme d'image Docker, et livré en production de manière automatisée par le pipeline **GitHub Actions**.

Ce rendu final est la preuve tangible que notre écosystème Cloud-Native est fonctionnel de bout en bout.

Chapitre 9

Conclusion Générale

Ce projet a permis de valider techniquement la conception, le déploiement et l'administration d'une infrastructure cloud-native complète. En s'appuyant sur l'état de l'art du mouvement DevOps, l'intégralité du cycle de vie du logiciel a été automatisée.

L'apport le plus significatif de ce travail réside dans la résolution de problèmes d'ingénierie réels induits par les contraintes matérielles du Free Tier AWS. Comme démontré lors de l'initialisation de Kubeadm, la limitation stricte de mémoire (1 Go) impose une réflexion architecturale profonde (création de Swap via Ansible, bascule vers K3s allégé).

Aujourd'hui, grâce à l'implémentation du pipeline CI/CD sur GitHub Actions, toute modification du code source applicatif est auditable, testée, compilée et livrée en production de manière transparente et sécurisée en quelques secondes.